



Universidade do Estado do Rio de Janeiro

Departamento de Engenharia de Sistemas e Computação

Arquitetura de Sistemas Operacionais – Turma 1

Trabalho 1 – Implementando um Simulador de Escalonamento

**Giulia Campos
Petro Liporace**

Trabalho apresentado à disciplina de Arquitetura de Sistemas Operacionais, ministrada pela Prof.^a Rafaela Brum.

Rio de Janeiro
23 de agosto de 2026

Sumário

1	Introdução	3
2	Objetivo	3
3	Premissas do Simulador	4
3.1	Limite de processos	4
3.2	Unidade de tempo	5
3.3	Tempo de CPU	5
3.4	Filas de prioridade	5
3.5	Preempção	6
3.6	Operações de entrada e saída	6
4	Desenvolvimento do Simulador	7
4.1	Representação dos processos	7
4.2	Estados dos processos	8
4.3	Implementação inicial sem entrada e saída	8
4.4	Admissão dos processos	9
4.5	Seleção do processo	9
4.6	Controle do quantum	9
4.7	Rebaixamento entre as filas	10
5	Validação do Algoritmo MLFQ	10
6	Cálculo do Turnaround	12
7	Adição das Operações de Entrada e Saída	13
7.1	Representação dos eventos de E/S	13
7.2	Contadores independentes	14
7.3	Ordem de tratamento após uma unidade de CPU	14
8	Geração Automática dos Processos	14
8.1	Geração do tempo de serviço	15

8.2	Geração dos eventos de E/S	15
8.3	Seleção aleatória do dispositivo	16
9	Registro dos Eventos da Simulação	16
9.1	Log dos processos criados	17
9.2	Log durante a execução	17
10	Fluxo Geral da Simulação	18
11	Segundo Algoritmo de Escalonamento	19
11.1	Premissas específicas	19
11.2	Critério de seleção	20
11.3	Preempção	20
11.4	Ordem de tratamento em cada unidade de tempo	20
11.5	Validação do Algoritmo SRTF	21
11.6	Comparação entre os dois algoritmos	22
12	Resultados da Simulação	24
13	Conclusão	26

1 Introdução

O escalonamento de processos é uma das atividades fundamentais realizadas por um sistema operacional. Em um sistema multiprogramado, diversos processos podem estar simultaneamente prontos para utilizar o processador, sendo necessário definir qual deles terá acesso à CPU em determinado instante.

O escalonador é responsável por realizar essa seleção de acordo com um algoritmo previamente definido. Dependendo da estratégia adotada, aspectos como prioridade, ordem de chegada, tempo de execução e quantum disponível podem influenciar diretamente a ordem na qual os processos são executados.

Além dos períodos de utilização da CPU, um processo pode realizar operações de entrada e saída (E/S). Nesses casos, o processo deixa temporariamente o processador e passa ao estado bloqueado enquanto aguarda a conclusão da operação solicitada. Durante esse período, outro processo pode utilizar a CPU.

Este trabalho apresenta o desenvolvimento de um simulador de escalonamento de processos. A implementação foi construída inicialmente considerando somente a utilização da CPU e, posteriormente, adicionando preempção, diferentes níveis de prioridade, operações de entrada e saída, geração automática de processos e acompanhamento dos eventos ocorridos durante a simulação.

O simulador implementa dois algoritmos. O primeiro é o de filas multiníveis com retroalimentação, exigido pelo enunciado. O segundo é o SJF preemptivo, que decide de maneira oposta usando o tempo de execução de cada processo, informação que o primeiro não tem. Rodar os dois sobre os mesmos processos permite comparar as duas estratégias.

2 Objetivo

O objetivo principal do trabalho foi desenvolver um simulador capaz de representar, de forma didática, o funcionamento de algoritmos de escalonamento utilizados por sistemas operacionais.

Entre os algoritmos considerados encontra-se obrigatoriamente o escalonamento por filas multiníveis com retroalimentação, também conhecido como *Multilevel Feedback Queue* (MLFQ).

Como segundo algoritmo foi implementado o SJF preemptivo, ou *Shortest Remaining Time First* (SRTF), descrito na seção 11.

O simulador foi projetado de forma que seja possível acompanhar o estado dos processos e das diferentes filas ao longo de cada unidade de tempo da simulação.

Foram considerados como objetivos específicos:

- representar os principais dados e os estados assumidos por um processo durante sua execução;
- implementar filas de diferentes níveis de prioridade;
- implementar preempção baseada em quantum;
- implementar operações de entrada e saída;
- permitir a execução simultânea conceitual da CPU e dos dispositivos de E/S;
- gerar automaticamente processos com características aleatórias;
- registrar os eventos ocorridos durante a simulação;
- calcular o tempo de turnaround de cada processo;
- implementar um segundo algoritmo de escalonamento e compará-lo com o primeiro sobre os mesmos processos.

As seções 6 a 10 descrevem componentes compartilhados pelos dois algoritmos: o cálculo do turnaround, o tratamento das operações de entrada e saída, a geração automática de processos, o registro dos eventos e o fluxo geral da simulação. Apenas as seções 4, 5 e 11 tratam de um algoritmo específico.

3 Premissas do Simulador

Antes da implementação do simulador, foram estabelecidas algumas premissas para definir de forma clara o comportamento esperado do sistema.

Essas premissas também permitiram simplificar determinados aspectos de um sistema operacional real, mantendo o foco nos conceitos de escalonamento apresentados durante a disciplina.

3.1 Limite de processos

O simulador adota um limite máximo de 10 processos por execução.

A quantidade efetivamente criada é informada por argumento de linha de comando e, quando omitida, são gerados 3 processos.

Esse limite evita que a saída passo a passo se torne extensa demais para acompanhamento, mantendo o simulador legível para fins didáticos.

3.2 Unidade de tempo

O simulador trabalha com unidades discretas de tempo.

A cada iteração do laço principal da simulação ocorre o avanço de uma unidade de tempo global.

Durante uma mesma unidade de tempo podem ocorrer diferentes atividades de forma conceitualmente paralela, como, por exemplo, um processo utilizando a CPU enquanto outro processo realiza uma operação de E/S.

3.3 Tempo de CPU

O tempo de CPU de um processo representa exclusivamente a quantidade de unidades de tempo nas quais ele precisa efetivamente utilizar o processador.

Portanto, o período em que um processo permanece bloqueado realizando uma operação de entrada e saída não reduz seu tempo restante de CPU.

Por exemplo, um processo que necessita de 10 unidades de CPU e realiza uma operação de E/S com duração de 5 unidades continua necessitando de apenas 10 unidades efetivas de processamento.

Caso seja o único processo existente no sistema e realize uma única E/S, seu tempo mínimo até a conclusão será:

$$10 + 5 = 15 \text{ unidades de tempo}$$

Entretanto, somente 10 dessas unidades correspondem à utilização da CPU.

3.4 Filas de prioridade

Para o algoritmo de filas multiníveis com retroalimentação foram utilizados três níveis principais:

Tabela 1: Filas utilizadas pelo algoritmo MLFQ

Fila	Prioridade	Quantum
Alta	0	2
Média	1	4
Baixa	2	6

A prioridade de valor numérico menor representa maior prioridade.

Todos os novos processos são inicialmente inseridos na fila de alta prioridade.

3.5 Preempção

Quando um processo utiliza integralmente o quantum disponível sem terminar sua execução ou solicitar uma operação de E/S, ele sofre preempção.

No algoritmo MLFQ adotado, o processo é então rebaixado para a fila imediatamente inferior.

Dessa forma:

$$Alta \rightarrow Media \rightarrow Baixa$$

Um processo que já esteja na fila de baixa prioridade e utilize todo o seu quantum retorna ao final da própria fila.

3.6 Operações de entrada e saída

Foram considerados três tipos de operação de entrada e saída:

Tabela 2: Tipos de entrada e saída considerados

Dispositivo	Duração adotada
Disco	3 u.t.
Fita	7 u.t.
Impressora	10 u.t.

Os valores representam o período durante o qual o processo permanece bloqueado aguardando a conclusão da respectiva operação.

O tempo de E/S é controlado separadamente do tempo de CPU.

Ao concluir a operação, a fila de prontos para a qual o processo retorna depende do dispositivo utilizado:

Tabela 3: Fila de retorno após cada tipo de operação

Dispositivo	Fila de retorno
Disco	Baixa prioridade
Fita	Alta prioridade
Impressora	Alta prioridade

O disco é rápido, então quem o usa se parece mais com um processo CPU-bound e volta para a fila baixa. Fita e impressora são lentas, típicas de processos I/O-bound, que ganham prioridade ao voltar.

Existe uma única fila de entrada e saída, atendida um processo por vez. Dessa forma, um processo pode permanecer aguardando na fila do dispositivo antes de iniciar sua operação.

4 Desenvolvimento do Simulador

O desenvolvimento foi dividido em etapas. Essa abordagem permitiu validar individualmente as principais partes do simulador antes da adição de novas funcionalidades.

A estratégia adotada foi começar com um caso simplificado contendo somente CPU e, posteriormente, aumentar progressivamente a complexidade da simulação.

4.1 Representação dos processos

O primeiro passo foi definir quais informações deveriam ser armazenadas para cada processo.

Para representar de forma simplificada, foram armazenados dados como:

- PID;
- PPID;
- instante de chegada;
- prioridade;
- estado atual;
- tempo total de CPU;
- tempo restante de CPU;
- quantidade de CPU já executada;
- quantum restante;
- fila de origem da execução atual;
- instante de término;

- eventos de entrada e saída.

A partir desses dados foi criada a classe **Processo**, que centraliza o estado de cada processo durante toda a simulação.

O campo de prioridade é mantido sempre coerente com a fila em que o processo se encontra. Sempre que ocorre um rebaixamento ou um retorno de entrada e saída, a prioridade registrada é atualizada.

Como o simulador não modela a criação de processos por outros processos, todos são criados diretamente pelo sistema e recebem PPID igual a zero.

4.2 Estados dos processos

Os estados foram representados através de uma enumeração.

Foram utilizados os seguintes estados:

NOVO: processo criado, porém ainda não admitido no sistema;

PRONTO: processo disponível para execução e aguardando uma CPU;

EXECUTANDO: processo atualmente utilizando a CPU;

BLOQUEADO: processo aguardando a conclusão de uma operação de E/S;

TERMINADO: processo que concluiu toda a sua execução.

A utilização de uma enumeração tornou explícitas as transições de estado realizadas durante a simulação.

4.3 Implementação inicial sem entrada e saída

Antes de inserir operações de E/S, foi implementada uma primeira versão contendo somente:

- chegada dos processos;
- filas de prioridade;
- seleção do processo;
- execução da CPU;
- controle de quantum;

- preempção;
- rebaixamento entre filas;
- término dos processos.

Essa decisão permitiu verificar isoladamente se o algoritmo de escalonamento estava correto.

4.4 Admissão dos processos

A cada unidade de tempo é verificado se algum processo possui instante de chegada igual ao tempo atual da simulação.

Quando isso ocorre, o processo altera seu estado de NOVO para PRONTO e é inserido na fila de alta prioridade.

4.5 Seleção do processo

Sempre que a CPU está livre, o escalonador procura um novo processo.

A busca respeita a seguinte ordem:

$$FilaAlta > FilaMedia > FilaBaixa$$

Assim, somente quando nenhuma tarefa está disponível na fila de alta prioridade a fila média é consultada.

Da mesma forma, a fila baixa somente é utilizada quando as duas anteriores estiverem vazias.

Dentro de cada fila, os processos seguem uma política FIFO.

4.6 Controle do quantum

Ao receber a CPU, o processo recebe o quantum correspondente à fila na qual estava armazenado.

A cada unidade de CPU utilizada:

- o tempo restante de CPU é decrementado;
- o tempo total já executado é incrementado;

- o quantum restante é decrementado.

De forma simplificada:

```
1 processo.tempo_restante -= 1
2 processo.tempo_cpu_executado += 1
3 processo.tempo_quantum -= 1
```

Quando o quantum chega a zero e o processo ainda possui tempo de CPU restante, ocorre uma preempção.

4.7 Rebaixamento entre as filas

No MLFQ utilizado, um processo que utiliza completamente a fatia de tempo disponível é transferido para um nível inferior.

O comportamento implementado segue:

```
1 se fila_atual == ALTA:
2     mover para MEDIA
3
4 senao se fila_atual == MEDIA:
5     mover para BAIXA
6
7 senao:
8     retornar ao final da fila BAIXA
```

Esse mecanismo permite favorecer inicialmente processos com rajadas curtas de CPU, enquanto processos que utilizam longos períodos de processamento são gradativamente deslocados para filas de menor prioridade.

5 Validação do Algoritmo MLFQ

Antes da inclusão das operações de entrada e saída, foi realizada uma validação utilizando um exemplo resolvido em sala de aula.

Foram considerados os seguintes processos:

Tabela 4: Processos utilizados para validação

Processo	Chegada	Tempo de CPU
P1	0	5
P2	0	4
P3	1	8
P4	4	6

Foram utilizadas filas com quantums iguais a 2, 4 e 6 unidades de tempo.

A sequência produzida pelo simulador foi:

Tabela 5: Sequência de execução obtida

Intervalo	Processo
0–2	P1
2–4	P2
4–6	P3
6–8	P4
8–11	P1
11–13	P2
13–17	P3
17–21	P4
21–23	P3

Os processos foram finalizados nos seguintes instantes:

Tabela 6: Tempos de término obtidos

Processo	Término
P1	11
P2	13
P3	23
P4	21

Os resultados foram iguais aos apresentados no exercício feito em sala, validando a lógica básica de chegada, seleção, quantum, preempção, rebaixamento e contagem do tempo global.

O segundo algoritmo passa pela mesma conferência, contra o exemplo do material da disciplina, na seção 11.5. A validação dele aparece mais adiante porque sua implementação vem depois, na seção 11.

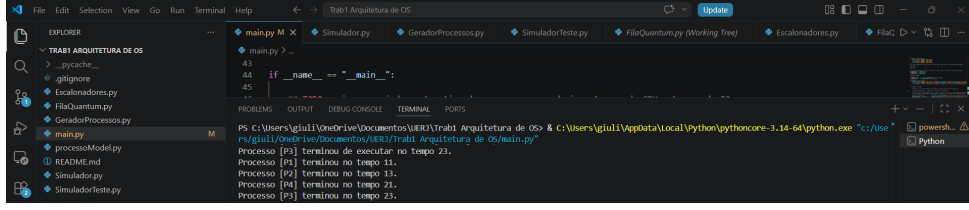


Figura 1: Resultado de teste do algoritmo MLFQ.

6 Cálculo do Turnaround

O turnaround corresponde ao intervalo total entre a chegada de um processo e sua conclusão.

Foi utilizada a expressão:

$$T_{turnaround} = T_{termino} - T_{chegada}$$

Para o teste inicial foram encontrados:

$$P1 = 11 - 0 = 11$$

$$P2 = 13 - 0 = 13$$

$$P3 = 23 - 1 = 22$$

$$P4 = 21 - 4 = 17$$

Portanto, o turnaround médio é:

$$\frac{11 + 13 + 22 + 17}{4} = 15,75$$

Esse resultado também foi utilizado como um dos critérios de validação da implementação.

7 Adição das Operações de Entrada e Saída

Após a validação do escalonamento sem E/S, a etapa seguinte consistiu em incorporar operações de entrada e saída.

A principal decisão tomada nessa etapa foi separar completamente o tempo de CPU do tempo utilizado pelos dispositivos de E/S.

Quando um processo solicita uma operação de E/S:

1. ele deixa a CPU;
2. seu estado passa para **BLOQUEADO**;
3. a operação é adicionada à respectiva fila de dispositivo;
4. a CPU pode selecionar outro processo;
5. o dispositivo de E/S é atualizado paralelamente ao processamento da CPU;
6. quando a operação termina, o processo volta ao estado **PRONTO**.

7.1 Representação dos eventos de E/S

Para permitir que um processo realize diversas operações de entrada e saída ao longo de sua execução, foi criada a estrutura **EventoIO**.

Cada evento contém:

- o tipo de dispositivo;
- a quantidade de CPU já executada necessária para disparar a operação;
- a duração da operação.

Por exemplo, um processo com 20 unidades de CPU pode possuir:

Tabela 7: Exemplo de eventos de E/S		
CPU executada	Operação	Duração
5	Disco	3
9	Fita	7
14	Impressora	10

Nesse caso, o processo não solicita E/S em tempos globais pré-determinados. O disparo depende da quantidade de CPU que o próprio processo conseguiu efetivamente executar.

Isso é necessário porque um processo pode sofrer diversas preempções antes de atingir o ponto de uma operação de E/S.

7.2 Contadores independentes

Foram mantidos contadores independentes para CPU e entrada e saída.

Enquanto o processo está executando:

```
1 tempo_restante -= 1
2 tempo_cpu_executado += 1
3 tempo_quantum -= 1
```

Enquanto está bloqueado em E/S:

```
1 tempo_io_restante -= 1
```

Assim, uma operação de E/S nunca reduz o tempo restante de CPU do processo.

7.3 Ordem de tratamento após uma unidade de CPU

Após cada unidade de CPU, foi definida uma ordem para verificar os eventos associados ao processo.

A prioridade adotada foi:

1. verificar se o processo terminou;
2. verificar se foi atingido um evento de E/S;
3. verificar se o quantum foi completamente utilizado.

Essa ordem evita, por exemplo, que um processo terminado seja recolocado desnecessariamente em uma fila de prontos.

8 Geração Automática dos Processos

Após a implementação da estrutura dos processos e dos eventos de entrada e saída, foi criado um gerador automático de processos.

O objetivo dessa etapa foi evitar que a simulação dependesse exclusivamente de processos definidos manualmente.

O gerador recebe parâmetros que estabelecem limites para a simulação, como:

- quantidade máxima de processos;
- quantidade máxima de eventos de E/S por processo;
- tempo mínimo de CPU;
- tempo máximo de CPU;
- menor instante possível de chegada;
- maior instante possível de chegada.

8.1 Geração do tempo de serviço

Para cada processo é sorteado um valor dentro do intervalo definido para o tempo total de CPU.

Por exemplo:

$$T_{CPU} \in [10, 30]$$

Dessa forma, diferentes processos podem apresentar diferentes necessidades de processamento.

8.2 Geração dos eventos de E/S

A quantidade de operações de E/S também pode variar entre os processos.

Se o número máximo permitido for três, um processo pode possuir:

$$0, 1, 2 \text{ ou } 3$$

eventos.

Os pontos de disparo são selecionados dentro do intervalo:

$$1 \leq T_{disparo} < T_{CPU}$$

Isso garante que nenhuma operação seja programada depois do término do processo.

Também são utilizados pontos distintos, impedindo que duas solicitações diferentes de E/S sejam disparadas exatamente após a mesma unidade de CPU.

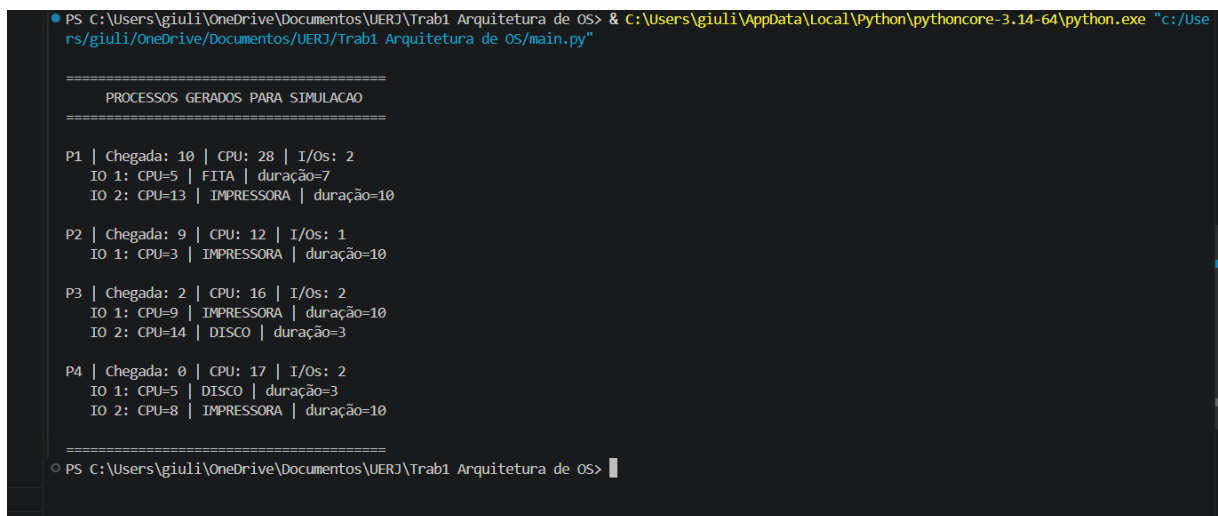
Depois da geração, os eventos são ordenados de acordo com seu ponto de disparo.

8.3 Seleção aleatória do dispositivo

Cada evento recebe aleatoriamente um dos tipos de E/S disponíveis:

- disco;
- fita;
- impressora.

A partir do dispositivo escolhido é determinada a duração da operação.



```
PS C:\Users\giuli\OneDrive\Documentos\UERJ\Trab1 Arquitetura de OS> & C:\Users\giuli\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/giuli/OneDrive/Documentos/UERJ/Trab1 Arquitetura de OS/main.py"

=====
PROCESSOS GERADOS PARA SIMULACAO
=====

P1 | Chegada: 10 | CPU: 28 | I/Os: 2
  IO 1: CPU=5 | FITA | duração=7
  IO 2: CPU=13 | IMPRESSORA | duração=10

P2 | Chegada: 9 | CPU: 12 | I/Os: 1
  IO 1: CPU=3 | IMPRESSORA | duração=10

P3 | Chegada: 2 | CPU: 16 | I/Os: 2
  IO 1: CPU=9 | IMPRESSORA | duração=10
  IO 2: CPU=14 | DISCO | duração=3

P4 | Chegada: 0 | CPU: 17 | I/Os: 2
  IO 1: CPU=5 | DISCO | duração=3
  IO 2: CPU=8 | IMPRESSORA | duração=10

=====
PS C:\Users\giuli\OneDrive\Documentos\UERJ\Trab1 Arquitetura de OS>
```

Figura 2: Log dos processos aleatórios gerados

9 Registro dos Eventos da Simulação

Um dos objetivos do simulador é permitir que o funcionamento do algoritmo seja acompanhado de forma didática.

Por esse motivo, foram adicionadas mensagens de log tanto no início quanto durante a execução.

9.1 Log dos processos criados

Antes do início do escalonamento, são apresentadas informações de todos os processos gerados.

Entre as informações exibidas estão:

- PID;
- instante de chegada;
- tempo total de CPU;
- quantidade de eventos de E/S;
- instante de disparo de cada E/S em relação à CPU executada;
- dispositivo utilizado;
- duração da operação.

Um exemplo de saída é apresentado a seguir:

```
1 P1 | Chegada: 2 | CPU: 20 | I/Os: 3
2
3 IO 1:
4 CPU = 4
5 Tipo = DISCO
6 Duracao = 3
7
8 IO 2:
9 CPU = 9
10 Tipo = FITA
11 Duracao = 7
12
13 IO 3:
14 CPU = 14
15 Tipo = IMPRESSORA
16 Duracao = 10
```

9.2 Log durante a execução

Durante a simulação são printados no console eventos como:

- chegada de um processo;
- entrada de um processo na CPU;
- execução de uma unidade de CPU;
- término do quantum;
- preempção;
- mudança de fila;
- solicitação de E/S;
- início da operação de E/S;
- término da operação de E/S;
- retorno do processo à fila de prontos;
- término de um processo.

Também são apresentadas as filas existentes e o processo que ocupa a CPU durante a unidade de tempo observada.

10 Fluxo Geral da Simulação

Após a integração das etapas anteriores, a lógica principal do simulador pode ser resumida pela sequência:

1. verificar a chegada de novos processos;
2. atualizar os dispositivos de entrada e saída;
3. verificar operações de E/S concluídas;
4. recolocar na fila de prontos os processos liberados pelas operações de E/S;
5. verificar se a CPU está disponível;
6. executar o escalonador caso seja necessário selecionar um novo processo;
7. executar uma unidade de CPU;
8. verificar término do processo;
9. verificar solicitação de entrada e saída;

10. verificar fim do quantum;
11. realizar preempção e mudança de fila quando necessário;
12. registrar o estado do sistema;
13. avançar o relógio global em uma unidade.

A simulação continua enquanto existir ao menos um processo que ainda não tenha atingido o estado **TERMINADO**.

11 Segundo Algoritmo de Escalonamento

O enunciado pede dois algoritmos, sendo um deles obrigatoriamente o de filas multiníveis com retroalimentação.

Como segundo algoritmo foi escolhido o SJF preemptivo, também chamado de *Shortest Remaining Time First* (SRTF).

Os dois trabalham de maneira oposta. O SRTF precisa saber de antemão quanto tempo de CPU cada processo vai consumir. O MLFQ não sabe nada disso e descobre durante a execução, rebaixando quem gasta o quantum inteiro. No fim, os dois acabam favorecendo os processos curtos.

11.1 Premissas específicas

O enunciado permite usar uma única fila quando o segundo algoritmo não tem níveis de prioridade. Foi o caso aqui:

- uma única fila de prontos;
- sem quantum: o processo só perde a CPU por preempção, ao pedir E/S ou ao terminar;
- o tipo de dispositivo não muda a fila de retorno, porque só existe uma fila;
- a escolha é feita pelo tempo de CPU restante.

O gerador de processos, os dispositivos e suas durações são os mesmos do MLFQ, para que os dois algoritmos possam ser comparados sobre os mesmos processos.

11.2 Critério de seleção

Com a CPU livre, é escolhido o processo com o menor tempo de CPU restante na fila de prontos.

```
1 processo = min(fila_prontos.fila,  
2               key=lambda p: p.TempoCpuRestante())
```

O critério usa o tempo restante de CPU, não o tempo total. O tempo total inclui as E/S ainda pendentes, que são espera em dispositivo; somá-las seria comparar tempo de processador com tempo de espera.

Havendo empate, fica o processo que entrou antes na fila.

11.3 Preempção

A preempção é checada logo depois das chegadas. É ali que pode aparecer um processo com tempo restante menor que o do processo em execução.

A troca só acontece se o tempo do candidato for estritamente menor:

$$T_{restante}(candidato) < T_{restante}(executando)$$

A comparação precisa ser estrita. Com $\leq$, dois processos empatados ficariam trocando de lugar na CPU a cada unidade de tempo sem nenhum dos dois avançar.

11.4 Ordem de tratamento em cada unidade de tempo

A cada unidade de tempo o simulador faz, nesta ordem:

1. admissão dos processos que chegam no instante atual;
2. verificação de preempção;
3. seleção de um processo, caso a CPU esteja livre;
4. processamento da fila de entrada e saída;
5. execução de uma unidade de CPU.

A E/S é tratada depois da seleção, igual ao MLFQ. Se viesse antes, um processo que acabasse a E/S poderia pegar a CPU no mesmo instante e gastar uma unidade de E/S e uma de CPU ao mesmo tempo.

11.5 Validação do Algoritmo SRTF

Como no MLFQ, a implementação foi conferida contra um exemplo resolvido no material da disciplina, sem E/S. Os processos usados foram:

Tabela 8: Processos utilizados para validação do SRTF

Processo	Chegada	Tempo de CPU
P1	0	8
P2	1	4
P3	2	9
P4	3	5

A sequência produzida pelo simulador foi:

Tabela 9: Sequência de execução obtida com o SRTF

Intervalo	Processo
0–1	P1
1–5	P2
5–10	P4
10–17	P1
17–26	P3

P1 é preemptado no instante 1, quando P2 chega com tempo de CPU restante menor.

Os processos foram finalizados nos seguintes instantes:

Tabela 10: Tempos de término obtidos com o SRTF

Processo	Término
P1	17
P2	5
P3	26
P4	10

Aplicando a expressão do turnaround:

$$P1 = 17 - 0 = 17$$

$$P2 = 5 - 1 = 4$$

$$P3 = 26 - 2 = 24$$

$$P4 = 10 - 3 = 7$$

Portanto, o turnaround médio é:

$$\frac{17 + 4 + 24 + 7}{4} = 13$$

Os valores batem com os do material, confirmando o critério de seleção e a preempção.

```
Windows PowerShell

PS I:\...\Trabalho_1_ArqSisOp\SJFPreemptivo> python main.py # exemplo do slide 11

Processo [P1] Inicializou na fila de prontos no tempo 0.
Processo [P1] foi selecionado com 8 u.t. de CPU restante.

Processo [P1] está executando no intervalo [0, 1]. CPU restante: 7. Tempo restante (CPU+E/S): 7.
Processo [P2] Inicializou na fila de prontos no tempo 1.
Processo [P1] sofreu preempção de [P2] e voltou para a fila de prontos.
Processo [P2] foi selecionado com 4 u.t. de CPU restante.

Processo [P2] está executando no intervalo [1, 2]. CPU restante: 3. Tempo restante (CPU+E/S): 3.
Processo [P3] Inicializou na fila de prontos no tempo 2.
Processo [P2] está executando no intervalo [2, 3]. CPU restante: 2. Tempo restante (CPU+E/S): 2.
Processo [P4] Inicializou na fila de prontos no tempo 3.
Processo [P2] está executando no intervalo [3, 4]. CPU restante: 1. Tempo restante (CPU+E/S): 1.
Processo [P2] está executando no intervalo [4, 5]. CPU restante: 0. Tempo restante (CPU+E/S): 0.
...

=====
TURNAROUND DOS PROCESSOS
=====

PID  Chegada  CPU  Término  Turnaround
P1    0        8    17        17
P2    1        4     5         4
P3    2        9    26        24
P4    3        5    10         7

Turnaround médio: 13.00 u.t.
=====
```

Figura 3: Saída do simulador para o exemplo de validação do SRTF.

11.6 Comparação entre os dois algoritmos

Os dois algoritmos foram executados sobre os mesmos processos, em dois perfis de carga: um com processos curtos e mais E/S, outro com processos longos e pouca E/S.

Tabela 11: Comparação entre MLFQ e SRTF

Métrica	MLFQ	SRTF
Turnaround médio (E/S intensa)	43,7	36,3
Turnaround médio (E/S reduzida)	111,5	75,1
Tempo de resposta (E/S intensa)	1,6	6,7
Tempo de resposta (E/S reduzida)	1,8	27,0

O SRTF ganha no turnaround nos dois perfis, porque atende primeiro quem termina antes. O MLFQ ganha no tempo de resposta, porque todo processo novo entra na fila de maior prioridade e pega a CPU quase na hora.

O preço do SRTF é starvation: um processo longo pode ficar esperando para sempre se processos curtos continuarem chegando. No MLFQ isso não acontece, porque mesmo o processo da fila mais baixa é atendido quando as de cima esvaziam.

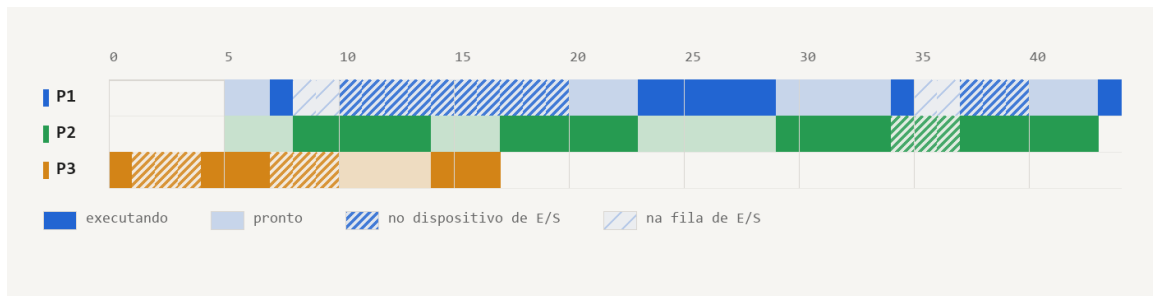


Figura 4: Gráfico de Gantt da execução com o MLFQ.

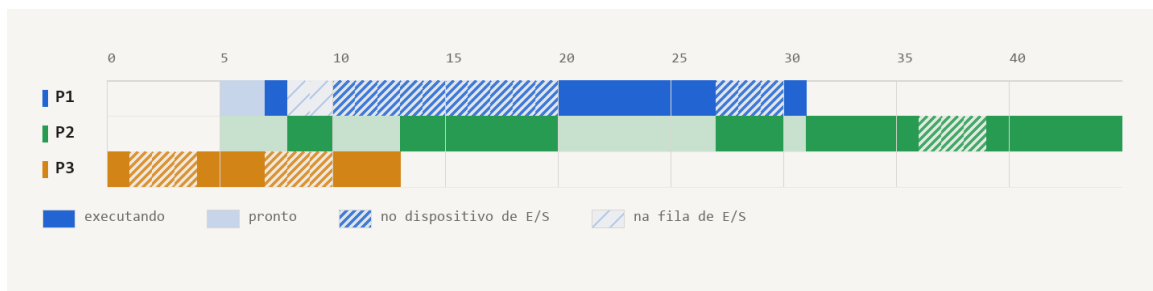
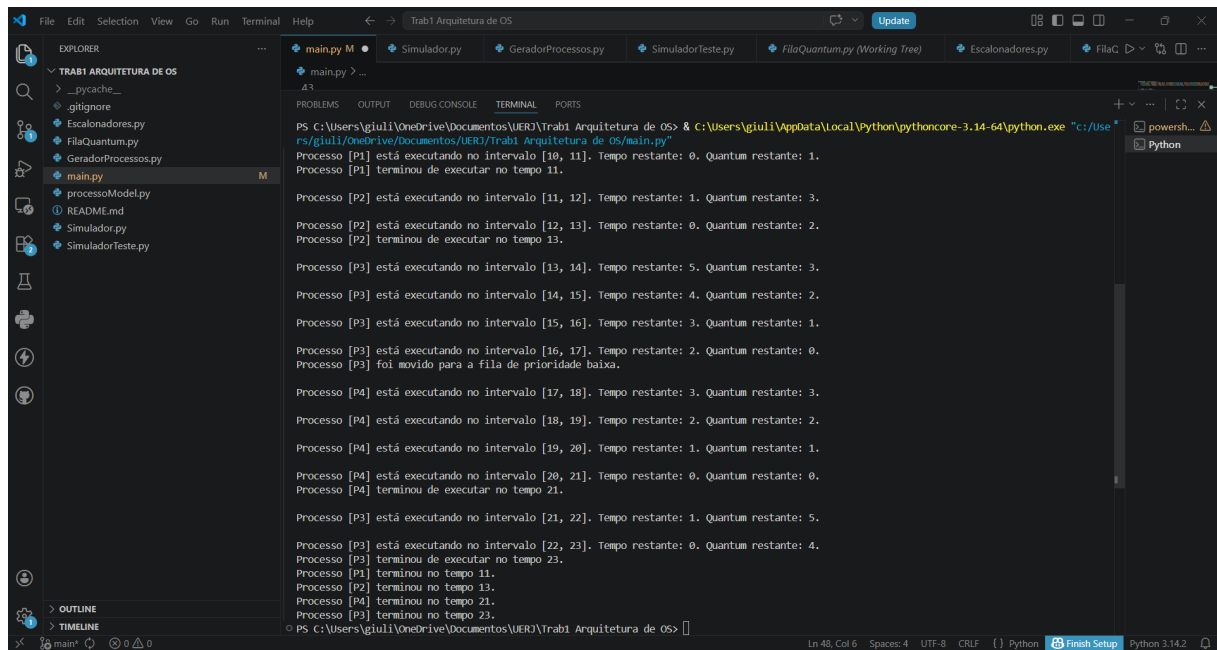


Figura 5: Gráfico de Gantt da execução com o SRTF, sobre os mesmos processos.

12 Resultados da Simulação



```
PS C:\Users\giuli\OneDrive\Documents\UERJ\Trab1 Arquitetura de OS> & C:\Users\giuli\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/giuli/OneDrive/Documents/UERJ/Trab1 Arquitetura de OS/main.py"
Processo [P1] está executando no intervalo [10, 11]. Tempo restante: 0. Quantum restante: 1.
Processo [P1] terminou de executar no tempo 11.

Processo [P2] está executando no intervalo [11, 12]. Tempo restante: 1. Quantum restante: 3.
Processo [P2] está executando no intervalo [12, 13]. Tempo restante: 0. Quantum restante: 2.
Processo [P2] terminou de executar no tempo 13.

Processo [P3] está executando no intervalo [13, 14]. Tempo restante: 5. Quantum restante: 3.
Processo [P3] está executando no intervalo [14, 15]. Tempo restante: 4. Quantum restante: 2.
Processo [P3] está executando no intervalo [15, 16]. Tempo restante: 3. Quantum restante: 1.
Processo [P3] está executando no intervalo [16, 17]. Tempo restante: 2. Quantum restante: 0.
Processo [P3] foi movido para a fila de prioridade baixa.

Processo [P4] está executando no intervalo [17, 18]. Tempo restante: 3. Quantum restante: 3.
Processo [P4] está executando no intervalo [18, 19]. Tempo restante: 2. Quantum restante: 2.
Processo [P4] está executando no intervalo [19, 20]. Tempo restante: 1. Quantum restante: 1.
Processo [P4] está executando no intervalo [20, 21]. Tempo restante: 0. Quantum restante: 0.
Processo [P4] terminou de executar no tempo 21.

Processo [P3] está executando no intervalo [21, 22]. Tempo restante: 1. Quantum restante: 5.
Processo [P3] está executando no intervalo [22, 23]. Tempo restante: 0. Quantum restante: 4.
Processo [P3] terminou de executar no tempo 23.
Processo [P1] terminou no tempo 11.
Processo [P2] terminou no tempo 13.
Processo [P4] terminou no tempo 21.
Processo [P3] terminou no tempo 23.
```

Figura 6: Passo a passo do processo sendo feito.

```
Windows PowerShell

PS I:\...\Trabalho_1_ArqSisOp\SJFPreemptivo> python main.py -p 3 -s 7

=====
PROCESSOS GERADOS PARA SIMULACAO
=====

P1 | Chegada: 5 | CPU: 9 | I/Os: 2
   IO 1: CPU=1 | IMPRESSORA | duração=10
   IO 2: CPU=8 | DISCO | duração=3

P2 | Chegada: 5 | CPU: 23 | I/Os: 1
   IO 1: CPU=17 | DISCO | duração=3

P3 | Chegada: 0 | CPU: 7 | I/Os: 2
   IO 1: CPU=1 | DISCO | duração=3
   IO 2: CPU=4 | DISCO | duração=3

=====
...

Processo [P3] está executando no intervalo [0, 1]. CPU restante: 6. Tempo restante (CPU+E/S): 12.
Processo [P3] solicitou E/S do tipo DISCO no tempo 1.
Processo [P3] foi movido para a fila de E/S.
Processo [P3] está realizando E/S do tipo DISCO no intervalo [1, 2]. Tempo restante de E/S: 2.
...
Processo [P3] voltou para a fila de prontos após E/S do tipo DISCO.
Processo [P2] está executando no intervalo [9, 10]. CPU restante: 21. Tempo restante (CPU+E/S): 24.
Processo [P2] sofreu preempção de [P3] e voltou para a fila de prontos.
Processo [P3] foi selecionado com 3 u.t. de CPU restante.

Processo [P1] está realizando E/S do tipo IMPRESSORA no intervalo [10, 11]. Tempo restante de E/S: 9.
Processo [P3] está executando no intervalo [10, 11]. CPU restante: 2. Tempo restante (CPU+E/S): 2.
```

Figura 7: Passo a passo da execução com o SJF preemptivo.

```
Windows PowerShell

PS I:\...\Trabalho_1_ArqSisOp\FilasMultiniveisComRetroalimentacao> python main.py -p 3 -s 7

=====
TURNAROUND DOS PROCESSOS
=====

PID  Chegada  CPU  Término  Turnaround
P1   5        9   44       39
P2   5       23   43       38
P3   0        7   17       17

Turnaround médio: 31.33 u.t.
=====
```

Figura 8: Tabela de turnaround impressa ao final da simulação.

Ao final da simulação, cada processo possui registrado seu instante de término.

A partir desse valor é possível calcular seu turnaround utilizando:

$$T_{turnaround} = T_{termino} - T_{chegada}$$

Os logs produzidos durante a execução também permitem acompanhar as mudanças de estado, a utilização das filas e as operações de entrada e saída realizadas por cada processo.

13 Conclusão

O desenvolvimento do simulador permitiu aplicar de forma prática os conceitos relacionados ao gerenciamento e ao escalonamento de processos em sistemas operacionais.

A estratégia de desenvolver inicialmente uma versão simplificada, contendo apenas processamento em CPU, mostrou-se útil para validar a lógica de filas, quantum e preempção antes da introdução de operações de entrada e saída.

A comparação da implementação inicial do MLFQ com um exemplo previamente resolvido permitiu verificar a correta evolução do tempo global e a ordem de execução dos processos.

Após essa validação, a separação entre tempo de CPU e tempo de E/S permitiu representar processos que alternam entre processamento e operações bloqueantes, mantendo a CPU disponível para outros processos durante esses períodos.

A criação de um gerador automático tornou possível produzir cenários diferentes de execução, variando tempos de chegada, tempos de CPU, quantidade de eventos de entrada e saída e dispositivos utilizados.

O segundo algoritmo permitiu comparar duas estratégias sobre os mesmos processos. O SJF preemptivo teve o menor turnaround médio nos dois perfis testados. O MLFQ teve tempo de resposta bem menor, entre quatro e quinze vezes conforme a carga.

Cada um serve a um objetivo diferente: o SRTF termina mais processos em menos tempo, o MLFQ dá CPU rápido a quem acabou de chegar e não deixa nenhum processo esperando para sempre.

O SRTF ainda depende de uma informação que um sistema operacional real quase nunca tem: quanto tempo cada processo vai executar. O MLFQ chega perto do mesmo resultado sem saber disso, apenas rebaixando quem gasta o quantum inteiro.

Por fim, os registros produzidos a cada unidade de tempo tornam a execução do simulador observável e facilitam a compreensão das decisões realizadas pelo escalonador.

Referências

- [1] BRUM, Rafaela. *Trabalho Prático 1 de Arquitetura de Sistemas Operacionais: Implementando um simulador de escalonamento*. Universidade do Estado do Rio de Janeiro, 2026.
- [2] BRUM, Rafaela. *Escalaonamento de Processos*. Material da disciplina Arquitetura de Sistemas Operacionais. Universidade do Estado do Rio de Janeiro, 2026.
- [3] BRUM, Rafaela. *Comunicação entre Processos*. Material da disciplina Arquitetura de Sistemas Operacionais. Universidade do Estado do Rio de Janeiro, 2026.
- [4] BRUM, Rafaela. *Exemplo de Escalonamento sem E/S*. Material de exercícios da disciplina Arquitetura de Sistemas Operacionais. Universidade do Estado do Rio de Janeiro, 2026.